Voici le résumé complet et structuré de l'architecture finale mise à jour pour ton agent IA :

Architecture Complète de l'Agent IA (Mise à jour)

1. NOUVEAU : Le Core Moteur & Sécurité (skills/core/)

Le cœur d'orchestration intègre désormais la gestion de mémoire, la sécurité et l'auto-correction :

- planner/ : Découpage des objectifs complexes en sous-tâches étapes par étapes.
- memory/ : Gestion de la mémoire à court terme (session) et long terme (base vectorielle / RAG).
- context-manager/ *(Token Budget)* : Compresse et nettoie dynamiquement le contexte pour éviter la saturation et réduire les coûts.
- router/ *(MCP Standard)* : Sélection automatique du bon skill au bon moment via le protocole MCP.
- auto-healing/ *(Validator + Reflection)* : Valide le résultat des actions et corrige automatiquement les erreurs en cas d'échec avant de répondre à l'utilisateur.
- guardrails/ *(Human-in-the-loop)* : Intercepte les actions critiques ou dangereuses (rm -rf, paiements, déploiements, envoi d'emails) pour demander une validation humaine.

2. Arborescence Rationalisée (8 Méta-Pôles)

Plaintext

3. Le Top 10 Indispensable Refondu

1. **Planner** — Découpage stratégique des tâches.
2. **Memory Manager** — Rétention du contexte et des informations clés.
3. **Context & Token Manager** — Optimisation et compression de la fenêtre de contexte.
4. **Tool Router** — Aiguillage automatique et standardisé (MCP).
5. **Web Research** — Recherche et synthèse d'informations fiables.
6. **Code Expert** — Génération, explication et correction de code.
7. **Terminal & Execution Sandbox** — Exécution isolée et sécurisée des commandes (ex: Docker / E2B).
8. **File System** — Gestion complète des fichiers et du code source.
9. **GitHub Manager** — Orchestration des dépôts, commits, PRs et versions.
10. **Auto-Healing & Validator** — Boucle d'auto-correction et validation des résultats.

4. Environnement d'Exécution (Point Clé)

- **Sandbox Obligatoire :** Toutes les commandes système et scripts du skill Terminal s'exécutent dans un conteneur éphémère et isolé pour protéger la machine hôte.

Si ton objectif est de créer **l'agent IA le plus puissant possible**, je ne garderais pas des milliers de skills. Je créerais plutôt un **ensemble de skills "intelligents"**, chacun capable de couvrir un domaine complet. C'est l'approche utilisée par les meilleurs agents actuels.

1. Core

Ces skills pilotent l'agent.

- Planner
- Orchestrator
- Task Manager
- Workflow Engine
- Goal Manager
- Context Manager
- Memory Manager
- Session Manager
- State Manager
- Reflection
- Self Critique
- Retry Manager
- Error Recovery

---

2. Recherche

- Web Search
- Deep Research
- Documentation Search
- GitHub Search
- API Search
- Stack Overflow Search
- Academic Search
- Semantic Search

---

3. Programmation

- Code Generation
- Code Review
- Refactoring
- Debugging
- Test Generation
- Performance Optimization
- Security Review
- Documentation Generator
- Architecture Design
- Dependency Analysis

---

4. Terminal

- Shell
- PowerShell
- Bash
- Command Runner
- Process Manager
- Package Manager
- Environment Manager

---

5. Git

- Git Workflow
- Repository Analysis
- Commit Generator
- Branch Manager
- Pull Request Review
- Merge Assistant
- Release Manager

---

6. Développement Web

- HTML
- CSS
- JavaScript
- TypeScript

- React
- Next.js
- Vue
- Tailwind
- Node.js
- Express
- FastAPI

---

7. Bases de données

- SQL
- PostgreSQL
- MySQL
- MongoDB
- Redis
- Prisma
- Migration Manager

---

8. DevOps

- Docker
- Docker Compose
- Kubernetes
- CI/CD
- GitHub Actions
- Terraform
- Nginx

---

9. Cloud

- AWS
- Azure
- GCP
- Cloudflare
- Vercel
- Railway
- Render

---

10. IA

- Prompt Engineering
- RAG
- Embeddings
- Vector Database
- Tool Calling
- Multi-Agent
- MCP
- Fine-tuning

---

11. Fichiers

- File Reader
- File Writer
- Project Analyzer
- File Search
- Diff Generator
- Patch Generator
- Archive Manager

---

12. Données

- CSV
- Excel
- JSON
- YAML
- XML
- PDF
- OCR
- Data Cleaning

---

13. Images

- Image Generation
- Image Analysis
- OCR
- Diagram Generator
- Screenshot Analysis

---

14. Audio/Vidéo

- Speech-to-Text
- Text-to-Speech
- Audio Analysis
- Video Analysis
- Subtitle Generator

---

15. Communication

- Email
- Documentation
- Translation
- Summarization
- Report Generator
- Presentation Generator

---

16. Sécurité

- Secret Scanner
- Vulnerability Scanner
- Dependency Audit
- JWT
- OAuth
- OWASP Review

---

17. Automatisation

- Browser Automation
- Playwright
- Puppeteer
- Selenium
- API Automation
- Workflow Automation

---

18. Monitoring

- Logs Analysis
- Metrics
- Profiling
- Benchmark
- Performance Reports

---

19. Productivité

- Calendar
- Todo
- Notes
- Kanban
- Meeting Summary
- Sprint Planner

---

20. Skills spécialisés

- Figma
- Notion
- Stripe
- Supabase
- Firebase
- OpenAI
- Anthropic
- Google AI
- Hugging Face
- Vercel
- Cloudflare

- Discord
- Slack
- Telegram

Architecture recommandée

Je recommande d'organiser les skills de cette manière :

```
skills/
│
├── core/
├── planning/
├── memory/
├── reasoning/
├── coding/
├── terminal/
├── filesystem/
├── git/
├── github/
├── web/
├── browser/
├── database/
├── devops/
├── cloud/
├── ai/
├── api/
├── automation/
├── security/
├── productivity/
├── media/
├── communication/
├── analytics/
├── monitoring/
└── integrations/
```

Les 10 skills indispensables

S'il fallait n'en garder que dix pour construire un excellent agent, je choisirais :

1. **Planner** — décompose les objectifs en étapes.
2. **Memory Manager** — conserve le contexte et les informations importantes.
3. **Reasoning** — choisit la meilleure stratégie pour résoudre un problème.
4. **Tool Router** — sélectionne automatiquement l'outil ou le skill adapté.
5. **Web Research** — recherche et synthétise des informations fiables.
6. **Code Expert** — écrit, corrige et explique du code.

7. **Terminal** — exécute des commandes et résout les erreurs.
8. **File System** — lit, modifie et organise les fichiers d'un projet.
9. **GitHub Manager** — gère les dépôts, les commits et les pull requests.
10. **Validator** — vérifie les résultats avant de les renvoyer à l'utilisateur.

Cette approche est beaucoup plus évolutive qu'une collection de milliers de petits skills, car chaque skill est spécialisé, réutilisable et facile à maintenir. Une fois cette base en place, tu peux ajouter des skills métier (finance, santé, design, etc.) sans modifier le cœur de l'agent.